

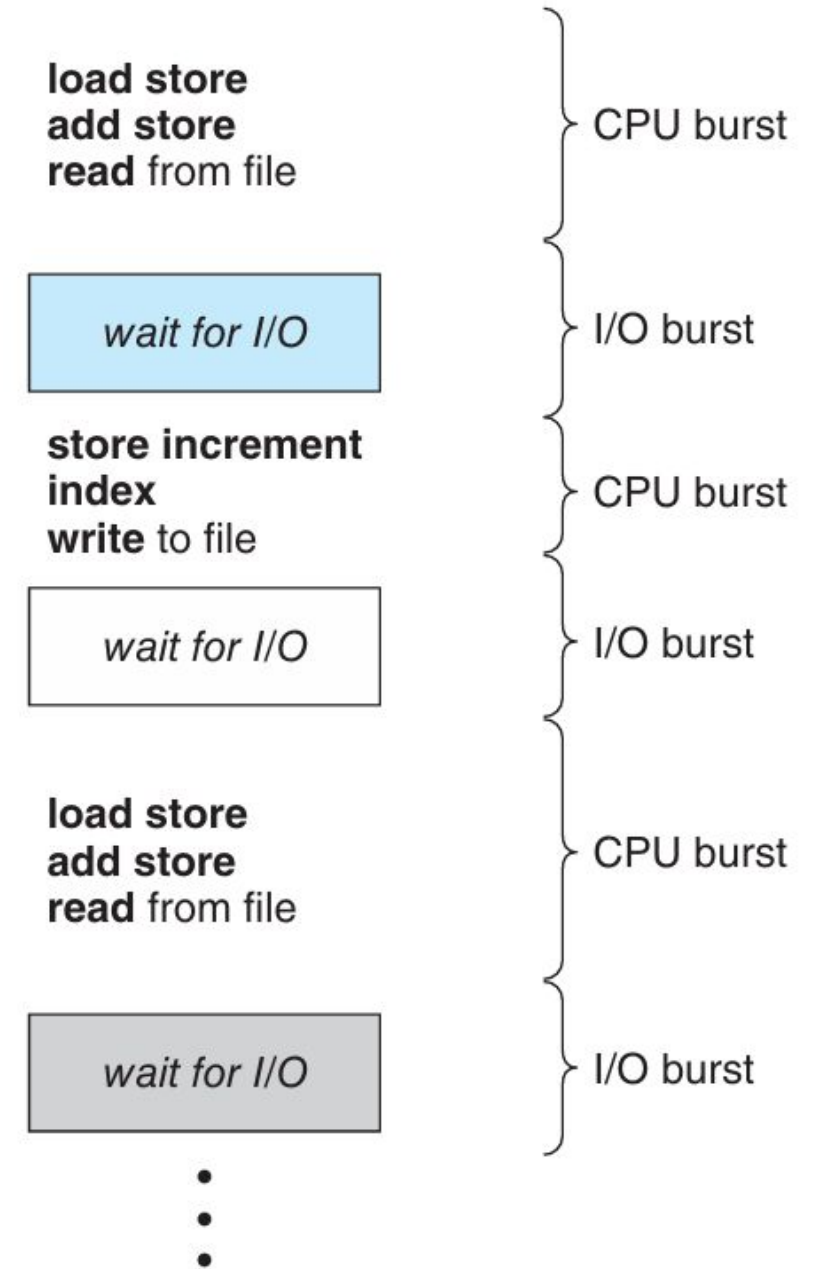


CPU SCHEDULING

Efficient use of the CPU

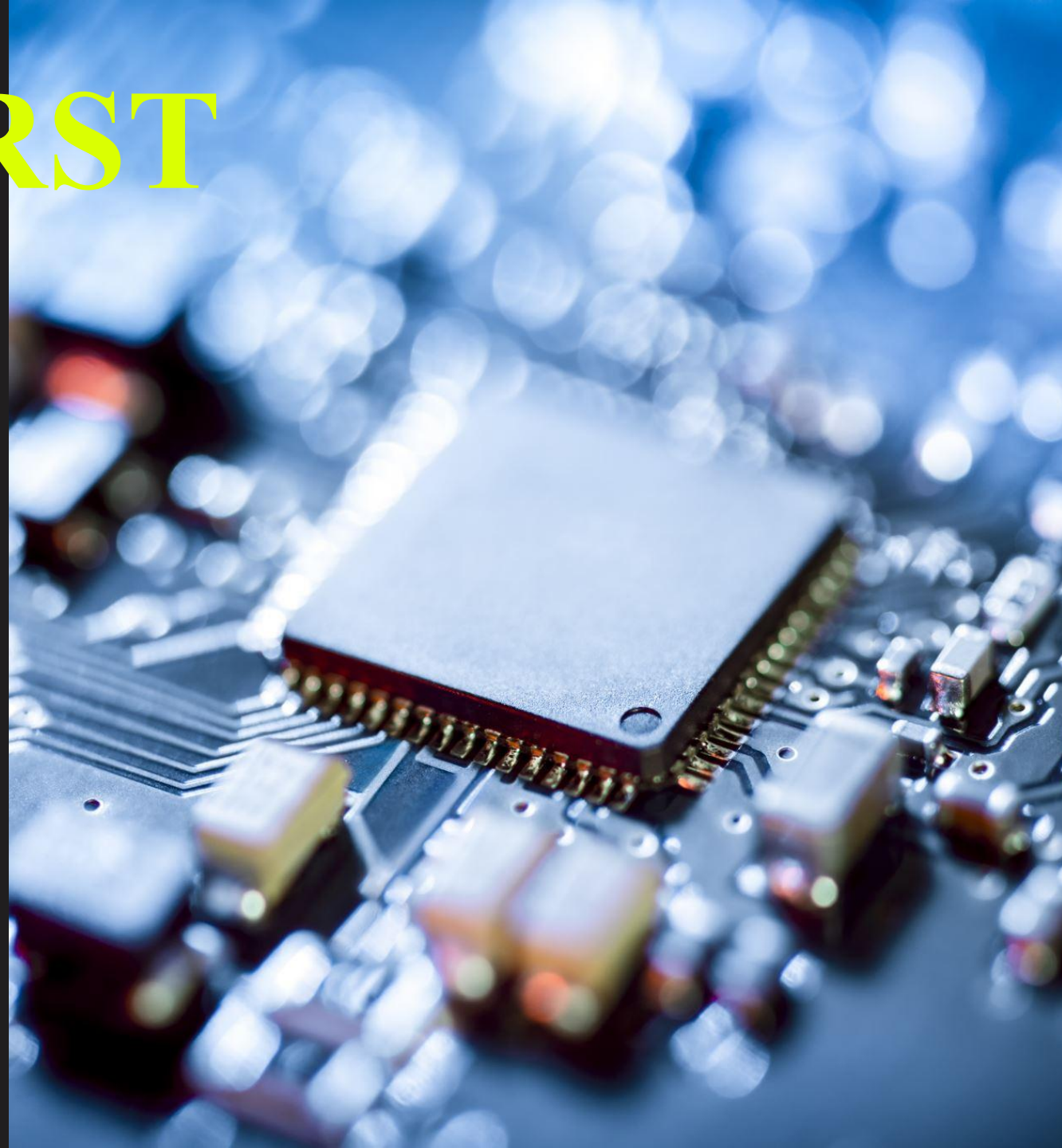
CPU-I/O BURST CYCLE

- Process execution follows a **repetitive cycle**
- Two phases:
 - **CPU Burst** → process executes on CPU
 - **I/O Burst** → process waits for I/O
- Execution pattern:
 - CPU burst → I/O burst → CPU burst → I/O burst → ...



CPU-I/O BURST CYCLE

- Process always starts with a **CPU burst**
 - During I/O burst:
 - Process is **blocked (waiting)**
 - CPU can be **allocated to another process**
- Cycle repeats **until completion**
 - Final step:
 - Ends with a **CPU burst**
 - Followed by a **system call for termination**



CPU SCHEDULING DECISION POINTS

- When process requests **I/O** or calls **wait()**
- CPU becomes **free**

- When a process is **interrupted**
- It is moved back to the **ready queue**

- When **I/O completes**
- Process is ready to **use CPU** again

- When a process **finishes execution**
- CPU is assigned to **another process**

PREEMPTIVE VS NON-PREEMPTIVE SCHEDULING

- **No Choice Situations (Forced Scheduling):**

- **1. Running → Waiting**

- **4. Process Termination**

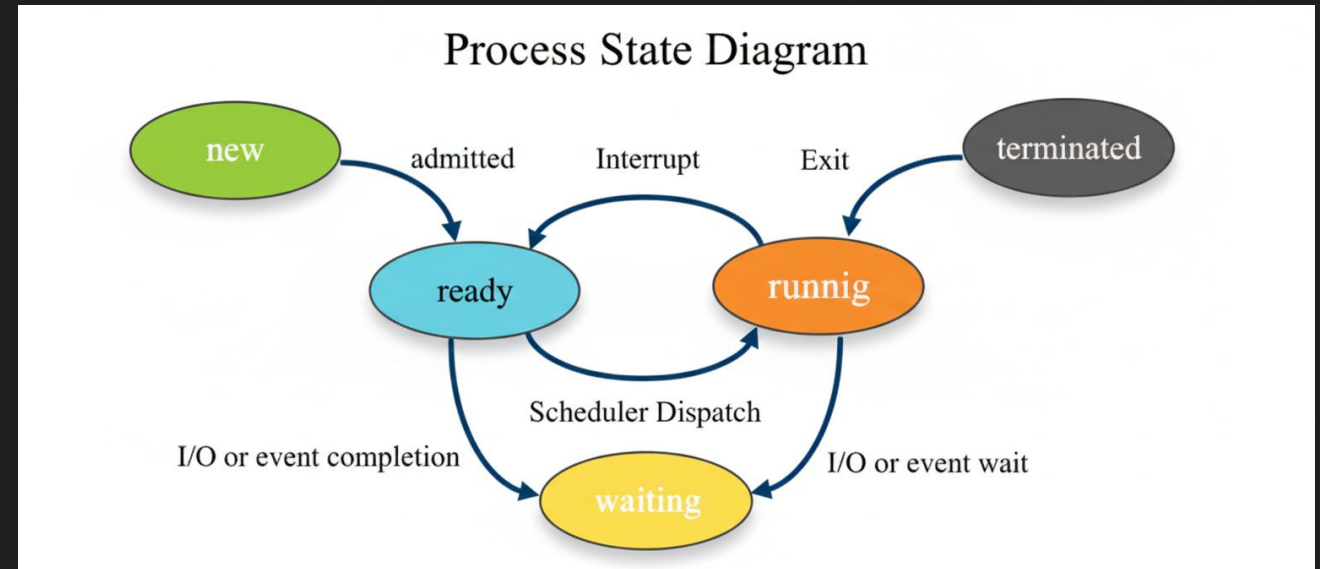
- **CPU must be assigned to another process**

- **Choice Situations:**

- **2. Running → Ready**

- **3. Waiting → Ready**

- **OS can decide whether to switch or continue**



PREEMPTIVE VS NON-PREEMPTIVE SCHEDULING

Non-Preemptive Scheduling (Cooperative)

- Scheduling occurs **only in cases 1 and 4**
- Once a process starts, it **runs until it waits or finishes**
- No interruption by OS

Preemptive Scheduling

- Scheduling can occur in **all four situations**
- OS can **interrupt a running process anytime**
- CPU can be **reassigned before completion**

IN A NUTSHELL

- Non-preemptive → **no interruption**
- Preemptive → **interrupt-driven control**
- Modern operating systems like Windows, macOS, Linux, and UNIX use **preemptive scheduling**

PREEMPTIVE VS NON-PREEMPTIVE SCHEDULING (PRECISE COMPARISON)

Aspect	Non-Preemptive Scheduling	Preemptive Scheduling
CPU Control	Process keeps CPU until it finishes or waits	OS can take CPU anytime
Interruption	 Not allowed	 Allowed
Decision Points	Only at termination or I/O wait	At any state change (all 4 cases)
Response Time	Slower (others may wait long)	Faster (quick response)
Complexity	Simple to implement	More complex
Efficiency	Can cause CPU idle or delays	Better CPU utilization
Use Case	Batch systems	Modern interactive systems

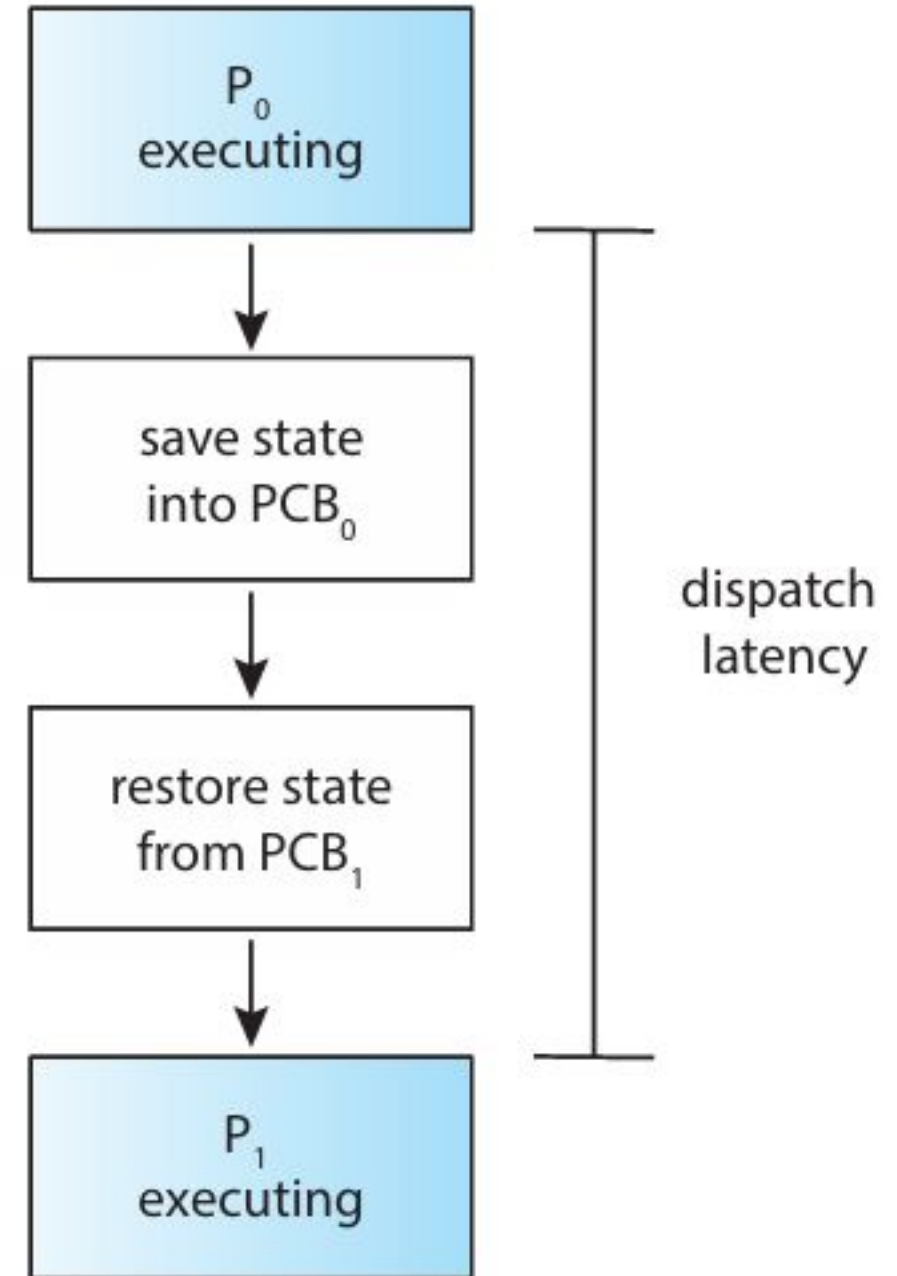
**AFTER THE OS
SELECTS A PROCESS,
WHO ACTUALLY GIVES
THAT PROCESS
CONTROL OF THE CPU?**

DISPATCHER

- The **dispatcher** is a key part of CPU scheduling
- It **hands over the CPU** to the process selected by the scheduler
- Its main functions:
 - **Context switching** → saves and loads process state
 - Switches CPU from **kernel mode to user mode**
 - Jumps to the **correct instruction in the process**

DISPATCH LATENCY

- Dispatcher should be **very fast**
- It runs during **every context switch**
- Responsible for **switching CPU between processes**
- **Dispatch latency:**
 - Time to **stop one process and start another**
 - Measures **context switch overhead**
- **Simple idea:**
 - **Scheduler decides who runs.**
 - **Dispatcher makes it happen on the CPU**



CPU SCHEDULING PERFORMANCE METRICS

CPU Utilization

Throughput

Turnaround time

Waiting Time

Response Time

CPU UTILIZATION

Percentage of time CPU is **busy**

Goal: keep CPU as close to **100% busy** as possible

Typically ranges: **40%** (light load) to **90%** (heavy load)

THROUGHPUT

Number of **processes completed per unit time**

Higher throughput =
more work done

Depends on process size (long vs short processes).

WAITING TIME

Total time a process spends in the **ready queue**

Does not include execution or I/O time

Directly affected by CPU scheduling algorithm

RESPONSE TIME

Time from **request submission** to **first response**

Important in **interactive systems**

Measures how quickly system **starts responding**, not full completion

TURNAROUND TIME

- Time from **process submission to completion**
- Includes:
 - Waiting time
 - CPU execution time

NOTATION AND CALCULATION

TAT = Turnaround Time

CT = Completion Time

AT = Arrival Time

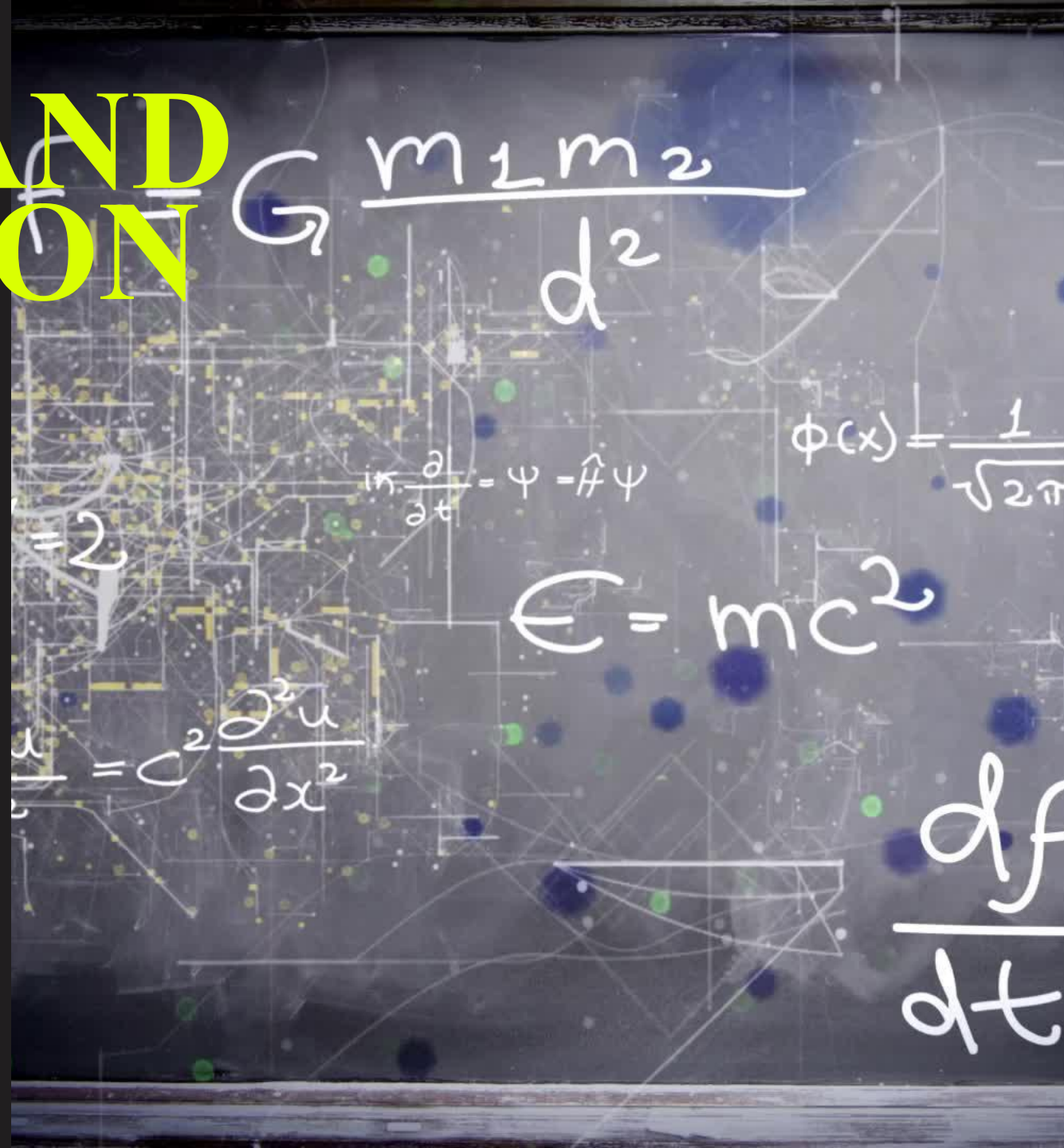
BT = Burst Time

AWT = Average Waiting Time

$TAT = CT - AT$

$WT = TAT - BT$

$RT = \text{First CPU Time} - AT$



SCHEDULING ALGORITHMS

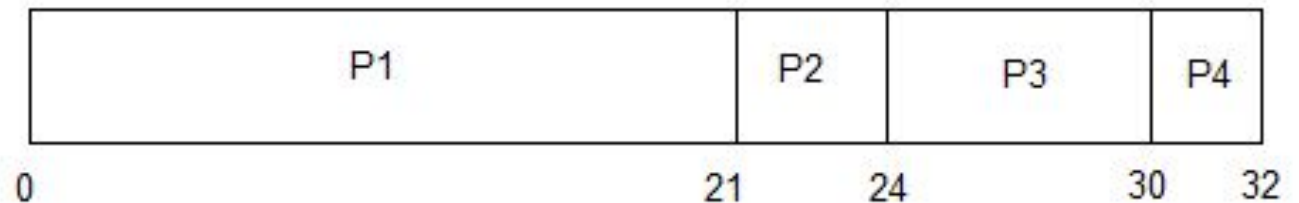
FIRST-COME, FIRST-SERVED SCHEDULING

- Process that arrives first is executed first
- Simple and easy to implement
- Used in simple batch processing systems
- Calculate TAT, CT and RT for each process

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



The average waiting time will be = $(0 + 21 + 24 + 30) / 4 = \underline{18.75 \text{ ms}}$



This is the GANTT chart for the above processes

TAT = CT - AT
WT = TAT - BT
RT = First CPU Time - AT

OVERALL RESULT

Process	BT	CT	TAT	WT	RT
P1	21	21	21	0	0
P2	3	24	24	21	21
P3	6	30	30	24	24
P4	2	32	32	30	30

SHORTEST JOB FIRST (SJF)

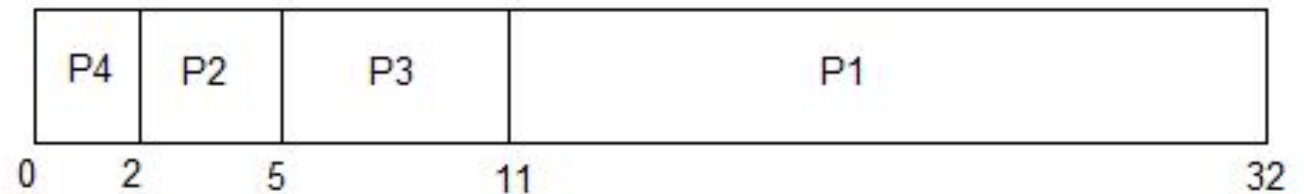
- Process with the **smallest CPU burst time** is selected first
- Reduces average waiting time
- Used in optimized job scheduling where burst time is known/estimated
- Calculate TAT, CT and RT for each process

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



In Shortest Job First Scheduling, the shortest Process is executed first.

Hence the GANTT chart will be following :



Now, the average waiting time will be = $(0 + 2 + 5 + 11)/4 = \underline{4.5 \text{ ms}}$

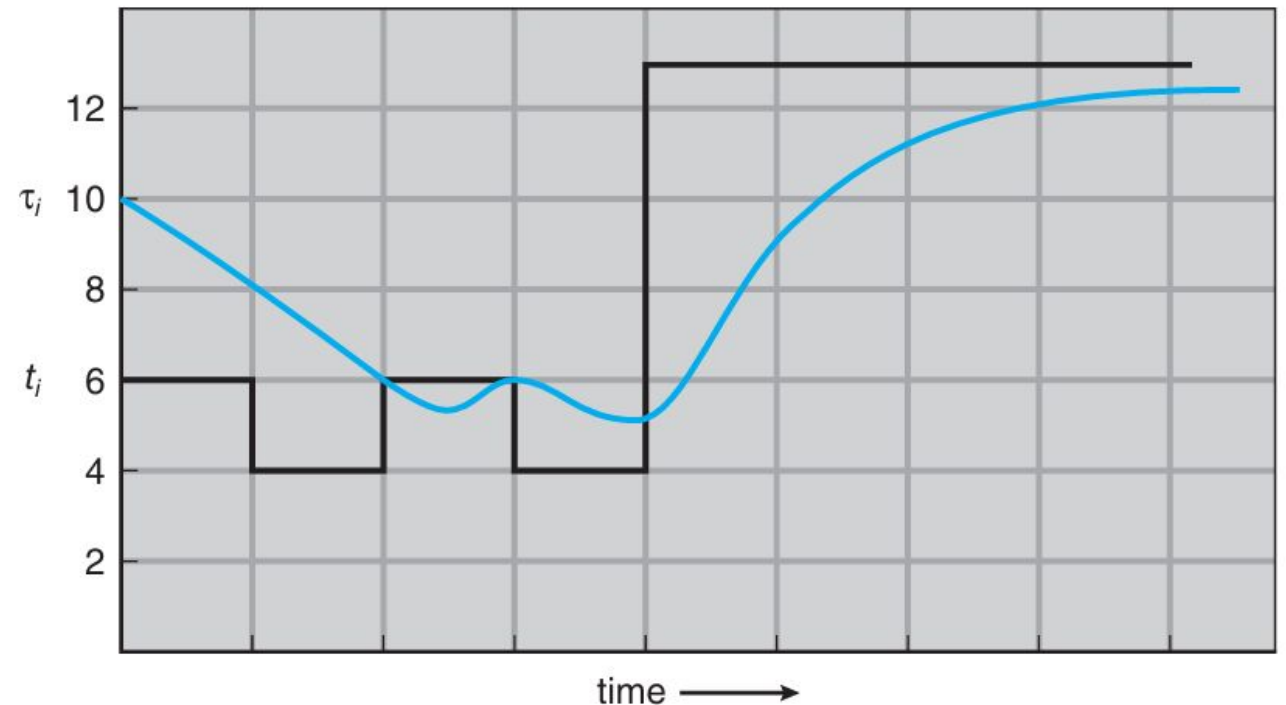
TAT = CT - AT
WT = TAT - BT
RT = First CPU Time - AT

OVERALL RESULT

Process	BT	CT	TAT	WT	RT
P4	2	2	2	0	0
P2	3	5	5	2	2
P3	6	11	11	5	5
P1	21	32	32	11	11

SJF LIMITATION AND APPROXIMATION

- SJF is **optimal but not directly implementable** at CPU level
- Reason: **next CPU burst time is unknown**
- Solution:
 - Estimate next burst using **past CPU burst history**
 - Assume future bursts are similar to previous ones
- Method used:
 - **Exponential averaging of previous CPU bursts**
- $\tau_{n+1} = \alpha \tau_n + (1-\alpha)t_n$ ($\alpha = 0.5$)



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

PREEMPTIVE VS NON-PREEMPTIVE SJF

Non-preemptive SJF

Preemptive SJF

- When a **new process** arrives in the ready queue while another is running

Non-preemptive SJF

- Current process is **not interrupted**
- Runs until its **CPU burst completes**

Preemptive SJF

- Current process is **interrupted** if new process has **shorter remaining time**
- Also called **Shortest Remaining Time First (SRTF)**

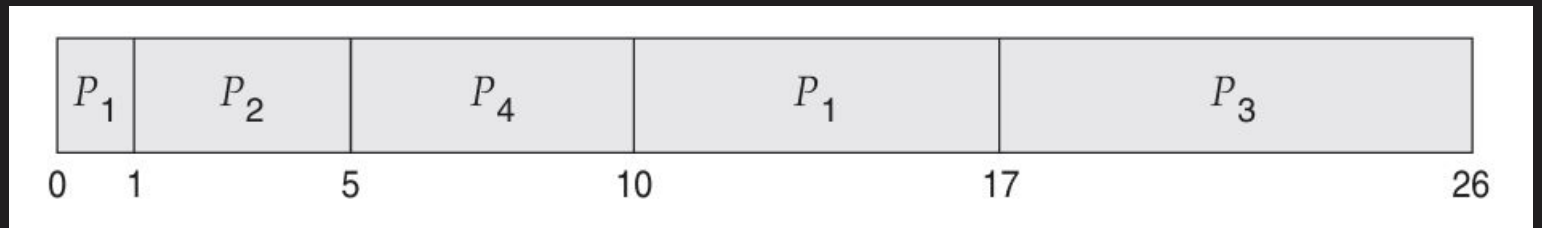
EXAMPLE

Data

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

$TAT = CT - AT$
 $WT = TAT - BT$
 $RT = \text{First CPU Time} - AT$

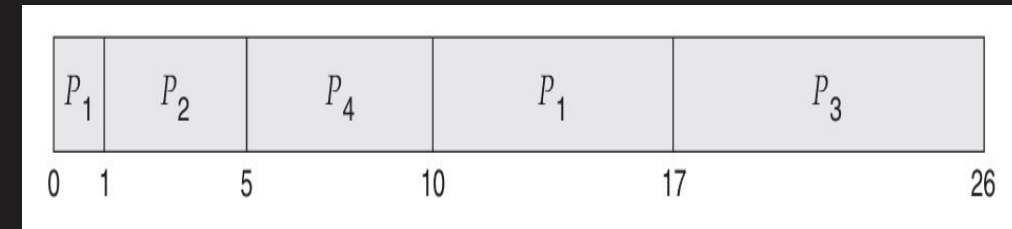
Grent Chart



CALCULATIONS

Grent Chart

Process	CT	Process	WT
P1	17	P1	$17 - 8 = 9$
P2	5	P2	$4 - 4 = 0$
P3	26	P3	$24 - 9 = 15$
P4	10	P4	$7 - 5 = 2$



$$AWT = \frac{9 + 0 + 15 + 2}{4} = \frac{26}{4} = 6.5$$

Process	TAT	Process	First Start	RT
P1	$17 - 0 = 17$	P1	0	$0 - 0 = 0$
P2	$5 - 1 = 4$	P2	1	$1 - 1 = 0$
P3	$26 - 2 = 24$	P3	17	$17 - 2 = 15$
P4	$10 - 3 = 7$	P4	5	$5 - 3 = 2$

ROUND-ROB IN SCHEDULING

Similar to **FCFS**, but with **preemption**.

Each process gets a fixed small time called **time quantum (time slice)**.

Time quantum is usually **10–100 ms**.

Ensures **fair CPU sharing** among all processes.

Used in time-sharing systems like Linux, UNIX, and Windows

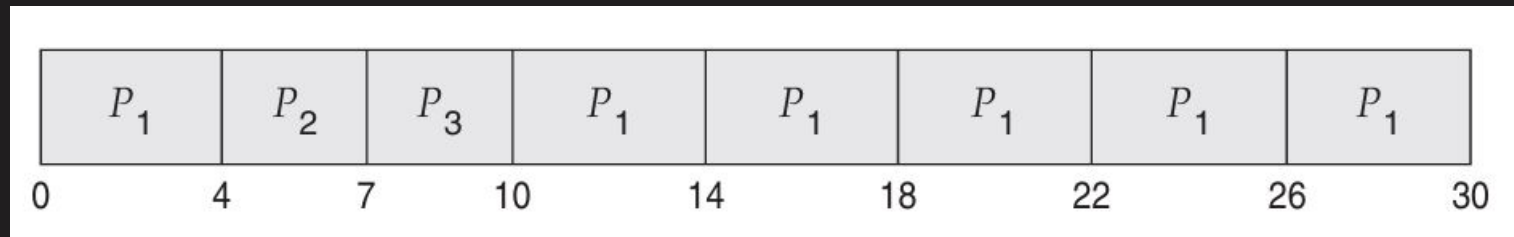
EXAMPLE

- Calculate Average Waiting Time for each process
- Calculate TAT, CT and RT for each process
- Quantum size = 4
- $AWT = (6+4+7) / 3 = 5.67$

Data

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

Grent Chart

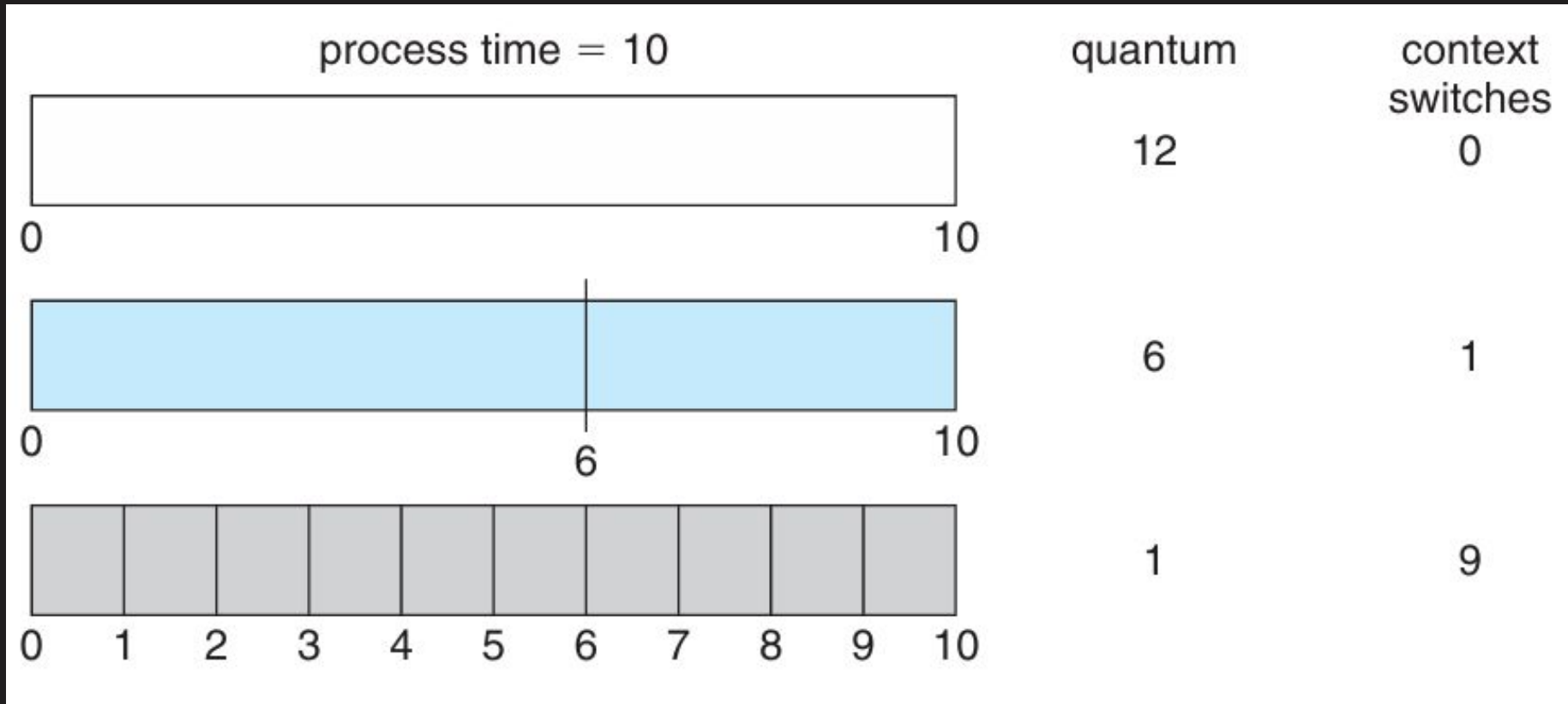


$TAT = CT - AT$
 $WT = TAT - BT$
 $RT = \text{First CPU Time} - AT$

OVERALL CALCULATIONS

Process	BT	CT	TAT	WT	RT
P1	24	30	30	6	0
P2	3	7	7	4	4
P3	3	10	10	7	7

QUANTUM SIZE AND CONTEXT SWITCHES

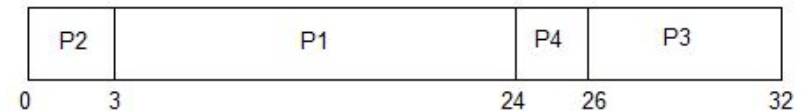


PRIORITY SCHEDULING

- A priority is associated with each process
- CPU is allocated to the process with highest priority
- Equal-priority processes are scheduled in FCFS order
- Used in real-time and system-critical task handling
- Calculate TAT, CT and RT for each process

PROCESS	BURST TIME	PRIORITY
P1	21	2
P2	3	1
P3	6	4
P4	2	3

The GANTT chart for following processes based on Priority scheduling will be,



The average waiting time will be, $(0 + 3 + 24 + 26) / 4 = \underline{13.25 \text{ ms}}$

TAT = CT – AT
WT = TAT – BT
RT = First CPU Time – AT

CALCULATION TABLE

Process	BT	Priority	CT	TAT	WT	RT
P2	3	1	3	3	0	0
P1	21	2	24	24	3	3
P4	2	3	26	26	24	24
P3	6	4	32	32	26	26

STARVATION IN PRIORITY SCHEDULING

- **Problem: Starvation (Indefinite Blocking)**
 - Low-priority processes may **never** get CPU
 - Continuous arrival of **high-priority** processes delays them **indefinitely**
- **Cause:**
 - Priority scheduling always favors **higher-priority** processes
- **Result:**
 - Some processes may wait **forever**
 - Extreme case: process may **never** execute

SOLUTION – AGING

Solution:

- Gradually **increase priority** of waiting processes over time
- Ensures long-waiting processes **eventually get CPU**

How it works:

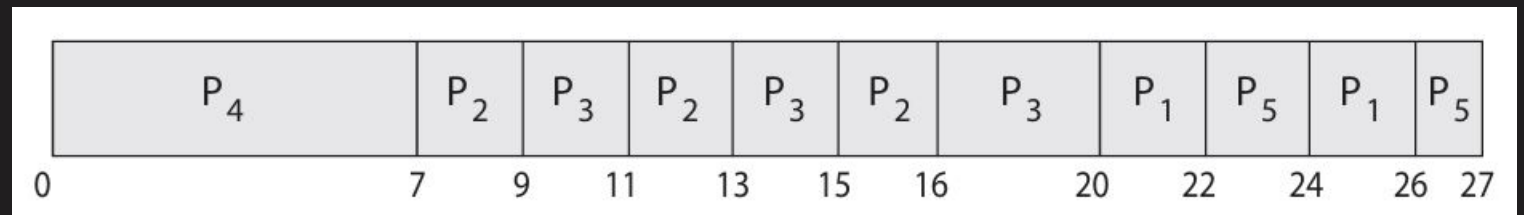
- Priority improves **step by step (e.g., every second)**
- A low-priority process will **reach highest priority eventually**

PRIORITY SCHEDULING WITH ROUND ROBIN

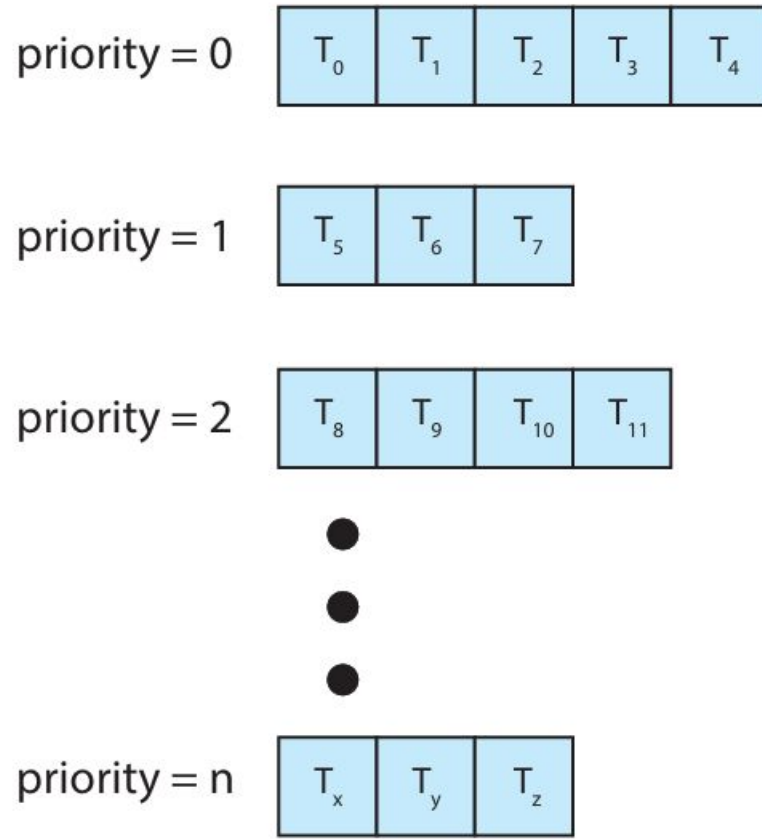
Data

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

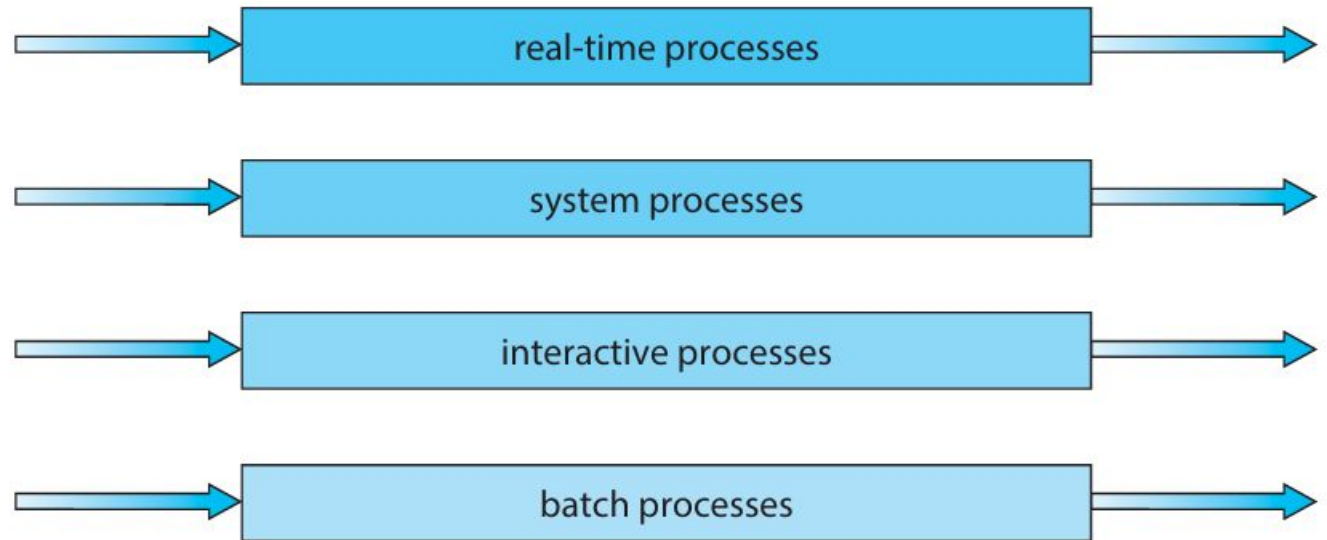
Grent Chart



MULTI-LEVEL QUEUEING



highest priority



lowest priority